



ENCRYPTACIÓN DE CONTRASEÑAS

Hash

CIFRAR O ENCRYPTAR CONTRASEÑA

Para mantener la seguridad de un sitio web, la contraseña que utilizan los usuarios para registrarse y autenticarse debe estar cifrada de tal forma que ni el administrador del sitio web ni el administrador de la base de datos puedan descifrarla, tampoco debe verse comprometida si hay un ataque a la base de datos

Usuario: Pepito

algoritmo matemático

(función hash)

Contraseña: pepito123

pepito123



417a9e5c9ac7c52e32727cfd25da99eca9339a80

ALGORITMOS MATEMÁTICOS Y FUNCIONES HASH DE PHP

Algoritmo matemático	Función PHP
MD5	md5()
SHA1	sha1()
SHA256	hash()
BLOWFISH	crypt()
	password_hash()

EL ALGORITMO IDEAL

Hay cuatro requisitos que un algoritmo de *hash* debe cumplir :

1. Debe ser *fácil* calcular el *hash* de una contraseña (mínimo tiempo posible).
2. Debe ser (computacionalmente hablando) en extremo difícil obtener la contraseña original a partir del *hash*, recibe el nombre de algoritmo irreversible
3. Los *hashes* generados por dos cadenas muy parecidas (digamos “Hola” y “hola”) deben ser absolutamente distintos, sin relación.
 - Por ejemplo, hash MD5 de Hola y hola es “f688ae26e9cfa3ba6235477831d5122e” y “4d186321c1a7f0f354b297e8914ab240”, respectivamente.
4. Debe ser inviable encontrar dos cadenas que generen el mismo hash (a esto se le llama *colisión*).

ALGORITMOS MATEMÁTICOS Y FUNCIONES HASH DE PHP

- MD5, SHA1 y SHA256 son algoritmos de encriptado de propósito general.
- El problema con utilizar MD5 y SHA1 para almacenar contraseñas es que el poder de cómputo moderno es bastante alto.
- A una computadora moderna le llevaría 40 segundos descifrar todos los hashes posibles si los usuarios utilizan contraseñas de seis dígitos alfanuméricas en minúsculas.
- Advertencia del manual de PHP sobre el uso de MD5 y SHA1
 - No se recomienda utilizar esta función para contraseñas seguras debido a la naturaleza rápida de este algoritmo de «hashing».
 - Véase las [Preguntas más frecuentes de «hash» de contraseñas](#) para más detalles y el empleo de mejores prácticas.

ALGORITMOS MATEMÁTICOS Y FUNCIONES

HASH DE PHP

PHP proporcionan una **API de hash de contraseñas nativa** que maneja cuidadosamente **el empleo de hash** y la **verificación de contraseñas** de una manera segura.

`password_hash()`

`password_verify()`

PASSWORD_HASH()

`password_hash()` crea un nuevo hash de contraseña usando un algoritmo de hash fuerte. Utilizará el algoritmo de que en ese momento se considere el más adecuado.

```
string password_hash(string $password, int $algo, array $options = []): string
```

```
//$algo: El algoritmo de hash a usar. Las opciones son PASSWORD_DEFAULT, PASSWORD_BCRYPT, y  
PASSWORD_ARGON2I (PHP 7.2) y PASSWORD_ARGON2ID (PHP 7.3).
```

```
//PASSWORD_DEFAULT: es un algoritmo de hash seguro actualmente recomendado por PHP.
```

```
<?php
```

```
$hash = password_hash($password, PASSWORD_DEFAULT);
```

```
?>
```

La longitud del resultado identificador puede cambiar con el tiempo, dependiendo del algoritmo de encriptación considerado por defecto en ese momento. Por lo tanto, se recomienda almacenar el resultado en una columna de una base de datos que pueda ampliarse a más de 60 caracteres (**256 caracteres sería una buena elección**).

PASSWORD_VERIFY()

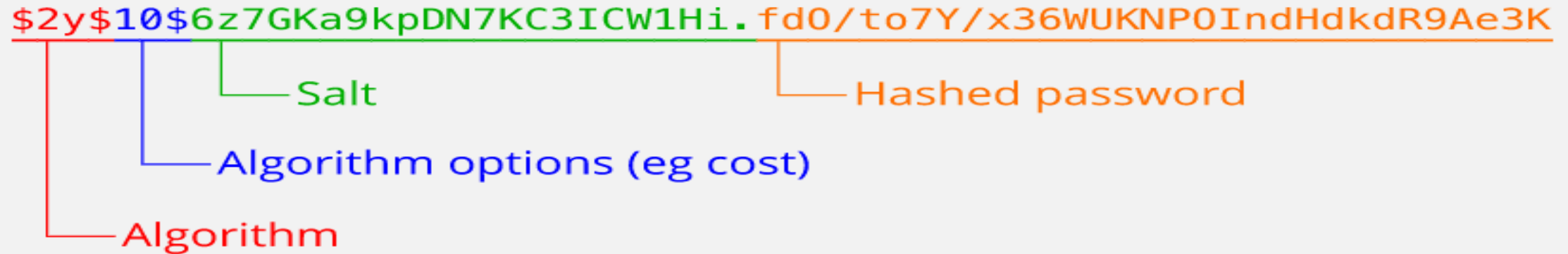
```
password_verify ( string $password , string $hash ) : bool
```

Comprueba que la contraseña coincida con el hash

Devuelve:

- **True:** si la contraseña y el hash coinciden
- **False:** si la contraseña y el hash no coinciden

¿QUÉ CONTIENE EL HASH GENERADO POR PASSWORD_HASH() ?



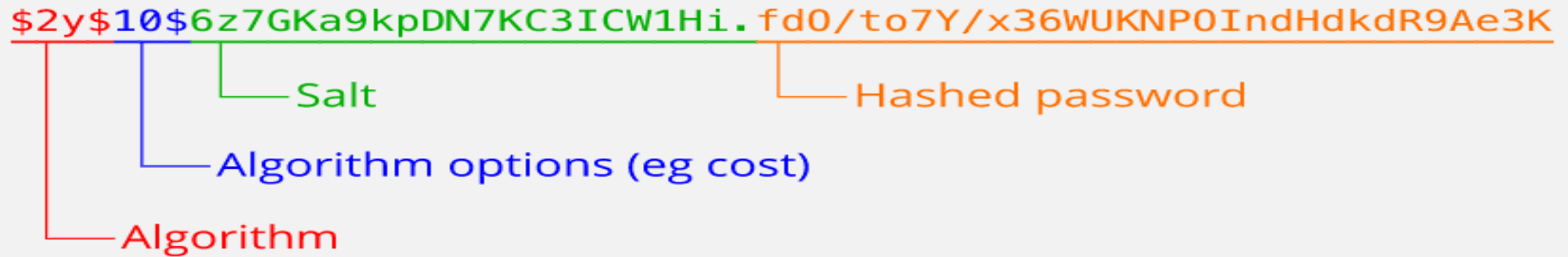
The diagram shows a password hash string: `$2y$10$6z7GKa9kpDN7KC3ICW1Hi.f d0/to7Y/x36WUKNP0IndHdkdR9Ae3K`. The string is divided into four color-coded sections with labels below them:

- Algorithm** (red): `$2y`
- Algorithm options (eg cost)** (blue): `$10`
- Salt** (green): `$6z7GKa9kpDN7KC3ICW1Hi.`
- Hashed password** (orange): `f d0/to7Y/x36WUKNP0IndHdkdR9Ae3K`

Una **salt** criptográfica es un dato que se utiliza durante el proceso de hash para eliminar la posibilidad de que el resultado pueda buscarse a partir de una lista de pares recalculados de hash y sus entradas originales, conocidas como **tablas rainbow**. Existe un gran número de servicios online que ofrecen grandes listas de códigos hash pre-calculados.

Una salt es un pequeño dato aleatorio añadido que hace que los *hashes* sean significativamente más difíciles de crackear. El uso de una salt hace muy difícil o imposible encontrar el hash resultante en cualquiera de estas listas.

¿QUÉ CONTIENE EL HASH GENERADO POR PASSWORD_HASH()



The diagram shows a password hash string: `$2y$10$6z7GKa9kpDN7KC3ICW1Hi.fD0/to7Y/x36WUKNP0IndHdkdR9Ae3K`. The string is divided into four color-coded sections with labels below them:
 - **Algorithm** (red): `$2y$`
 - **Algorithm options (eg cost)** (blue): `10$`
 - **Salt** (green): `6z7GKa9kpDN7KC3ICW1Hi.`
 - **Hashed password** (orange): `fD0/to7Y/x36WUKNP0IndHdkdR9Ae3K`

```
password_hash ( string $password , integer $algoritmo [, array $options ] ) : string
```

La función admite un array con opciones donde se puede establecer la salt y el coste en milisegundos invertido en general el hash.

Precaución: Se recomienda encarecidamente que no se genere una sal propia para esta función. Una sal segura se generará automáticamente si no se especifica una. Proporcionar la opción *salt* en **PHP 7.0** generará una advertencia de obsolescencia. El soporte para proporcionar una sal manualmente podría ser eliminado en una versión futura.


```
<?php
// Definir una contraseña para realizar pruebas
// Esta contraseña solo la conocerá el usuario, nunca se guardará en un BD
// Esta contraseña será la que introduzca el usuario en el formulario de login
// La aplicación nunca podrá recuperar la contraseña original
$password = "secreto123";

// Crear el hash de la contraseña usando password_hash()
$hash = password_hash($password, PASSWORD_DEFAULT);

// Mostrar el hash generado, este se guardará en la base de datos.
// NUNCA GUARDAR LA CONTRASEÑA EN TEXTO PLANO, SIEMPRE GUARDAR EL HASH
echo "El hash generado es: " . $hash . "<br>";

// Simulando el proceso de verificar la contraseña ingresada por el usuario
$usuario_ingresa_password = "secreto123"; // Contraseña correcta
$usuario_ingresa_password_erronea = "incorrecto123"; // Contraseña incorrecta

// Verificar si la contraseña introducida por el usuario coincide con el hash
// Esta función devuelve true si la contraseña es correcta, de lo contrario devuelve false
// Nunca devuelve ni el hash ni la contraseña original
$verificacion1 = password_verify($usuario_ingresa_password, $hash);
$verificacion2 = password_verify($usuario_ingresa_password_erronea, $hash);

// Mostrar los resultados de las verificaciones
echo "Verificación con contraseña correcta: " . ($verificacion1 ? 'Correcta' : 'Incorrecta') . "<br>";
echo "Verificación con contraseña incorrecta: " . ($verificacion2 ? 'Correcta' : 'Incorrecta') . "<br>";
?>
```